

ME301 Lab 3 Report

Group D: Edward Lee, Alicia Malek

3/15/2026

ABSTRACT

Machine learning methods allow robotic systems to adapt their behavior through experience rather than relying solely on manually programmed control rules. In this project, reinforcement learning was implemented to train a hexapod robot to maintain a following distance from a human target using sonar readings. A Q-learning algorithm was used in which the robot discretizes distance measurements into a finite set of states and selects among several motion actions while updating a Q-table through repeated interaction with its environment. Training experiments were conducted across multiple trials to evaluate the robot's ability to learn the desired following behavior. The results demonstrate that reinforcement learning can enable a robot to develop adaptive navigation behaviors while responding to real-time sensor feedback.

I. INTRODUCTION

Autonomous robots often operate in environments that are uncertain, dynamic, and difficult to model precisely. Traditional robotic control systems rely on predetermined rules or mathematical models to map sensor inputs to actuation commands. While this approach can be effective in structured and consistent environments, it becomes increasingly difficult to design and maintain as environmental complexity grows. As a result, many modern robotic systems incorporate machine learning techniques that allow robots to improve their performance through experience rather than relying exclusively on manually engineered control strategies.

One widely used form of machine learning in robotics is reinforcement learning. Reinforcement learning involves repeatedly selecting actions and receiving feedback in the form of rewards or penalties [1]. Through this process, the robot learns a control policy rather than having one manually programmed. As the robot continues to explore and evaluate different actions, it gradually develops a policy that maximizes the cumulative reward it receives. This approach is particularly well suited to robotics applications because it enables robots to learn behaviors through direct interaction with their environment without requiring labeled datasets or explicit supervision. For example, a robot navigating a maze could use reinforcement learning to determine which actions lead it closer to a goal state while avoiding negatively rewarded outcomes such as collisions with walls.

The specific reinforcement learning approach used in this lab is Q-learning. Q-learning is a model-free reinforcement learning algorithm that uses a state–action table to estimate the expected long-term reward associated with each possible action in a given state. After each action is performed, the corresponding value in the Q-table is updated based on the received reward and the estimated future rewards. These updates are performed using the Bellman equations shown below in Eq.1 [2]. The specific implementation of Q-learning used in this project will be discussed in detail in the Methods section.

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (1)$$

Q-learning is particularly useful for decision-making problems involving multiple states and actions in dynamic environments. One advantage of the algorithm is that it accounts for future rewards through a discount factor, allowing the robot to prioritize long-term outcomes rather than only immediate rewards [3]. This enables the learned policy to produce more stable and effective behaviors over time.

Another form of machine learning commonly used in robotics is supervised learning. Unlike reinforcement learning, supervised learning relies on an external source that provides labeled examples or corrective feedback. Instead of receiving reward signals, the robot receives an error signal that describes the difference between the predicted output and the correct output [1]. This approach is beneficial for tasks where the desired output is known and the error can be easily computed. For example, in robotic arm control tasks, the difference between a desired position and the actual position can be directly calculated and used to train the system.

A growing area of machine learning research is learning by demonstration. Humans and animals are able to learn new skills by observing others perform them. Researchers aim to replicate this process in robotic systems by allowing robots to observe demonstrations and infer the underlying behavior [1]. However, implementing this capability in robots is challenging because the system must determine what aspects of the observed behavior are important and how they should be translated into the robot's own motion and control commands. Despite these challenges, learning by demonstration has been successfully applied to tasks such as object manipulation, delivery tasks, and dynamic balancing behaviors.

Overall, machine learning provides a wide range of methods that enable robots to perform complex tasks in uncertain environments. These techniques allow robots to develop autonomous decision-making capabilities, improve perception, and adapt to changing conditions. Machine learning approaches are increasingly used in applications such as industrial automation, collaborative robots, medical robotics, and autonomous systems [4].

II. METHODS

In this lab, a hexapod robot was trained to follow a human or object while maintaining an optimal distance. The robot used a sonar sensor to measure the distance to the target and applied reinforcement learning with a Q-learning policy to determine its motion behavior. The desired behavior mimics that of a pet following a person. Specifically, the robot learns to move faster when it is farther from the target and slower when it is near the optimal following distance. If the robot loses detection of the target, it performs a search behavior to reacquire it. In this implementation, the search behavior consists of a right-turn motion.

The machine learning method used in this lab is reinforcement learning with a Q-learning algorithm. In this approach, a state-action Q-table is initialized with values of zero. During operation, the robot observes the distance to the target using the sonar sensor and categorizes the reading into discrete states representing different distance ranges. Each state represents a specific distance interval between the robot and the target. The defined state space is shown in Table 1. The robot can then choose among a set of predefined actions that determine its motion behavior, shown in Table 2.

Table 1. State Space for Q-Learning

States	Distance Range (mm)
(0) Very Close	<150
(1) Close	151-250
(2) Good	251-450
(3) Far	451-650
(4) Very Far	651-1500
(5) No Detection	>1500

Table 2. Action Space for Q-Learning

Actions	Behavior
0	Slow Forward
1	Fast Forward
2	Slow Backward
3	Fast Backward
4	Turn Right
5	Stop

The sonar sensor always faces the direction of the robot's heading, allowing it to continuously measure the distance to the target in front of the robot. The slow walking gaits are implemented as half the speed of the fast walking gaits. The Turn Right action rotates the robot approximately 9 degrees with each execution. The Stop action returns the robot to its neutral standing configuration.

The reinforcement learning process consists of two phases: training and testing. During the training phase, the robot is placed randomly within the environment at a detectable distance from the target. The target remains stationary during this phase to allow the robot to learn a stable distance-following behavior. The robot's initial position and orientation are randomized before each training episode. Training consists of 10 episodes, each containing 10 steps, where one step corresponds to a single action taken by the robot. This results in a total of 100 training steps.

Action selection during training uses an ϵ -greedy exploration strategy. The parameter ϵ represents the probability of selecting a random action rather than the best action currently stored in the Q-table. The ϵ value initially ranges from 0 to 1, where 1 corresponds to completely random action selection. After each training episode, ϵ decays by 10% until it reaches a minimum value of 0.05. As ϵ decreases, the algorithm gradually transitions from exploration toward exploitation. Exploitation refers to selecting the action that currently has the highest expected reward in the Q-table. After the robot determines its current state and selects an action, the action is executed. The sonar sensor is then read again to determine the new state of the robot relative to the target. A reward value is then assigned to the corresponding state-action pair based on the resulting state. These rewards are designed to encourage the robot to maintain the optimal following distance while discouraging undesirable behaviors such as losing the target or moving farther away from it. The reward structure used in the algorithm is summarized in Table 3.

Table 3. All Possible Rewards

State / Criteria	Reward Value	State / Criteria	Reward Value
Very Close	-20	Not Detected	-2
Close	-10	Before: Goal State ; After: Goal State	+10
Good	+40	Before: State (Not Goal) ; After: Same State (Not Goal)	-10
Far	-10	Before: No Detection ; After: Any Detection State	+60
Very Far	-20	Absolute Value of Distance Decreases after Action	+20
Absolute Value of Distance Increases after Action	-20		

After the reward is determined, the Q-table is updated using the Bellman equation, shown in Eq. (1). This update incorporates the received reward and the estimated value of future states to refine the expected value of the selected state-action pair. The parameters used for the Bellman equation are shown in Table 4.

Table 4. Bellman Equation Parameters and Significance

Parameter	Value	Significance
α (Learning Rate)	0.7	Higher the number, the greater the updated value
γ (Discount Rate)	0.9	Higher the number, the greater emphasis on future long-term rewards
r (Reward)	Variable	Cumulative reward for a singular taken action

After completing all 100 training steps, the final Q-table represents the learned policy for the robot's behavior. This table contains the estimated reward value for every possible state–action combination. The learned policy is then evaluated during the testing phase. During testing, the robot selects actions exclusively based on the highest-value action in the Q-table, without random exploration. The robot is again placed at random initial positions and orientations. Unlike the training phase, the target may move during testing to evaluate the robot's ability to maintain the desired following distance under dynamic conditions.

Several assumptions were made in order to simplify the learning environment and ensure effective training. First, it is assumed that the human or object being followed remains within the detectable range of the sonar sensor during operation. If the target temporarily moves outside this range, it is assumed that it will re-enter the detectable region so that the robot can reacquire it while performing its search behavior. When the robot loses detection, it continuously executes a right-turn motion to scan the environment until the target is detected again. Additionally, during the training phase, if the robot remains in the “No Detection” state for more than two consecutive actions, the training episode is terminated and the robot is considered lost. This assumption prevents the robot from accumulating excessive negative rewards due to prolonged loss of the target, which could otherwise bias the Q-table updates and degrade the learning process. Lastly, the nearby environment must be completely empty besides the tracked object, as the sensor has no method of differentiating objects.

Several performance metrics were recorded during both the training and testing phases to evaluate the effectiveness of the learning algorithm. During training, sonar distance readings, state transitions, selected actions, reward values, and Q-table updates were recorded at each step. These metrics are important because they allow the internal learning behavior of the algorithm to be verified and analyzed. Tracking these values helps confirm that the robot receives appropriate rewards when moving closer to the optimal distance and penalties when moving farther away, ensuring that the Q-learning algorithm is updating correctly. These values were recorded through program output during execution and by saving the final Q-table to a data file after training was completed. During testing, the performance of the learned policy was evaluated using the percentage of time the robot remained in the “Good” distance state, which represents the optimal following distance. This metric is important because it directly measures how effectively the robot maintains the desired distance from the target during operation.

III. RESULTS

The results from the first episode of the training are displayed below in Table 5. This episode quits prematurely due to being in a No Detection state for 2 steps and considers itself lost.

Table 5. Training Phase Data

Episode	Step	State	Next State	Action	Distance (mm)	Reward	Q Cell Update
1	0	1	1	5	243	-20	-14.38184
1	1	1	3	3	488	-10	-7.29850
1	2	3	4	2	670	-30	-21.908723
1	3	4	3	1	452	0	0.000000
1	4	3	4	2	721	-30	-27.32761
1	5	4	5	4	5000	-2	-1.489072

1	6	5	5	5	5000	-12	-8.32984
1	7	Quit	Quit	Quit	Quit	Quit	Quit

During the training phase, the robot spent 44% of the time in the optimal distance. On the other hand, with a finalized Q-table, the robot spent 78% of the time in the optimal distance during testing. This percentage was calculated by comparing the number of states in the “Good” state with the total number of steps.

After all 100 steps, the finalized Q-table was:

0	0	68.912	0	0	0
0	0	0	299.368	0	0
227.697	2.684	116.266	254.359	206.191	403.803
0	317.611	0	0	-25.69	-7
-14	59.049	0	-10.692	-16.799	-14
29.774	21.283	16.707	26.948	39,182	32.476

IV. DISCUSSION

The results demonstrate that the reinforcement learning algorithm improved the robot’s ability to maintain the desired following distance. During the training phase, the robot remained in the optimal “Good” state approximately 44% of the time. After the Q-table was finalized and the learned policy was applied during testing, the robot remained in the optimal state 78% of the time. This improvement indicates that the robot successfully learned which actions produced higher cumulative rewards and adjusted its behavior accordingly. Early in training, the robot frequently entered undesirable states such as “Very Far” or “No Detection,” resulting in negative rewards. As training progressed, the algorithm updated the Q-table using the Bellman equation, allowing the robot to gradually prioritize actions that reduced the distance error and maintained the target within the optimal range.

One limitation of the implementation involves how the robot responds when the target moves outside the detectable sonar range. Because the algorithm does not store the previous state or directional history, the robot cannot determine whether the target moved to its left or right. If both “Turn Left” and “Turn Right” actions were included, the robot could potentially oscillate between the two directions indefinitely or randomly select turning actions without effectively searching for the target. To simplify the control policy and avoid this issue, the robot was restricted to a single search behavior consisting of a right-turn motion when no object is detected. While this simplification prevents oscillatory behavior, it also limits the robot’s ability to efficiently reacquire the target if the target moved to the opposite side of the robot’s heading.

Another limitation arises from how the reward structure interacts with the “No Detection” state during training. The turn-right action is only rewarded appropriately if a single turn results in the robot transitioning from the “No Detection” state back to a detectable state. However, if multiple turns are required before the target is detected again, the robot receives repeated penalties for remaining in the “No Detection” state. In these cases, the algorithm may incorrectly associate the turning behavior with negative outcomes even though turning is necessary

for reacquiring the target. This makes the training process somewhat inconsistent and can lead to unusual behavior during testing when the robot attempts to search for the target.

Additionally, the relatively small number of training steps used in this experiment limits the total experience available for the reinforcement learning algorithm. The robot was trained for only 100 total steps, which restricts the number of state–action combinations the algorithm can explore, which resulted in several cells having a reward value of 0. Reinforcement learning systems typically benefit from significantly larger training datasets, allowing the Q-table values to converge toward more stable estimates. With more training episodes and greater exploration of the state space, the learned policy could potentially become more consistent and reliable.

The simplicity of the sensing system also introduces limitations in the operating environment. The robot relies solely on a forward-facing sonar sensor, meaning it cannot distinguish between different objects or environmental obstacles. For reliable operation, the surrounding environment must be relatively empty so that the sonar primarily detects the intended target. If additional objects are present within the sonar detection range, the robot may incorrectly interpret them as the target and attempt to follow them.

Several improvements could address these limitations. One possible improvement would be incorporating additional sensors or scanning mechanisms that allow the robot to determine the direction of the target relative to its heading. For example, a rotating sonar or vision-based sensor could provide directional information that enables the robot to select a more efficient turning direction when the target is lost. Another potential improvement would involve incorporating short-term state memory into the reinforcement learning model so that the robot can consider previous states when selecting actions. This would allow the algorithm to support multiple turning behaviors without risking oscillation. These enhancements could make the robot more robust in complex environments and improve its ability to maintain target tracking under dynamic conditions.

V. CONCLUSION

In this lab, reinforcement learning using a Q-learning algorithm was implemented to train a hexapod robot to follow a human or object while maintaining an optimal distance. The robot used sonar sensor measurements to classify the distance to the target into discrete states and selected motion actions based on a learned Q-table. Through repeated interaction with the environment and the use of a reward-based learning structure, the robot gradually learned which actions produced the most desirable outcomes. The ϵ -greedy exploration strategy allowed the robot to initially explore different behaviors and then transition toward exploiting the highest-value actions as the Q-table was updated.

The results demonstrated that the robot's performance improved after training. During the training phase, the robot remained within the optimal following distance 44% of the time, while during testing this value increased to 78%. This improvement indicates that the reinforcement learning algorithm was successful in developing a control policy that allows the robot to maintain the desired distance from the target.

However, several limitations were identified in the current implementation. The simplified search behavior, limited number of training steps, and reliance on a single forward-facing sonar sensor restrict the robot's ability to operate in more complex environments. Despite these limitations, the experiment demonstrates how reinforcement learning can enable robots to learn adaptive behaviors through interaction with their environment. Future improvements such as increased training duration, more advanced sensing capabilities, and improved search strategies could further enhance the robot's ability to track and follow moving targets in dynamic environments.

REFERENCES

- [1] “The robotics primer,” *Choice Reviews Online*, vol. 45, no. 06, pp. 45–322245–3222, Feb. 2008, doi: <https://doi.org/10.5860/choice.45-3222>.
- [2] GeeksforGeeks, “QLearning in Reinforcement Learning,” *GeeksforGeeks*, Feb. 06, 2019. <https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/>
- [3] “Q - Learning A basic form of Reinforcement Learning,” *Merkle.com*, 2022. <https://www.merkle.com/en/merkle-now/articles-blogs/2022/Feb/q-learning-a-basic-form-of-reinforcement-learning.html> (accessed Mar. 14, 2026).
- [4] “Machine learning in robotics can transform manufacturing,” *Autodesk.com*, 2022. <https://autodesk.com/design-make/articles/machine-learning-robotics> (accessed Mar. 14, 2026).